Designing Enterprise Retrieval-Augmented Generation (RAG) Systems

1. Introduction

AI models like GPT and Gemini are great at writing and answering questions. But they only know what they learned during training. They cannot read a company's private files. Retrieval-Augmented Generation, or RAG, fixes this. It lets an AI search a company's own documents first, then use that information to answer questions.

Companies use RAG because it is cheaper than retraining a model, and it keeps company data private. It also lets the AI show its sources, so people can trust the answer. Without RAG, an AI model can only guess. It may sound confident even when it is wrong. This is called hallucination, and it is one of the biggest problems with using AI alone.

2. Enterprise RAG Architecture

An enterprise RAG system has a few main parts, and each one does a specific job. The User Interface is what people type into, like a chat box. The Backend/API handles requests and connects everything together. Document Processing reads uploaded files and gets the text out. Chunking splits that text into small pieces. The Embedding Model turns each piece into numbers that capture its meaning. These numbers are stored in a Vector Database. When someone asks a question, the Retriever searches that database for the closest matches, and sends them to the LLM, which writes the final answer.

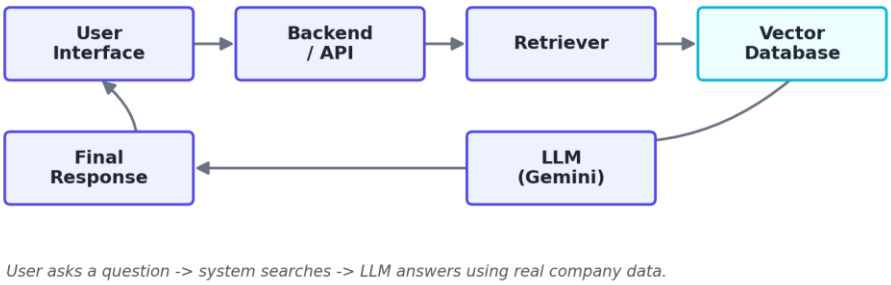


Figure 1: The main parts of an enterprise RAG system.

3. Embeddings

An embedding turns text into a list of numbers that captures its meaning. Text with similar meaning gets similar numbers, even if the words are different. This is why embeddings are useful: they let the system search by meaning, not just by matching exact words.

Popular embedding models include Sentence Transformers (like all-MiniLM-L6-v2), BGE, E5, and OpenAI Embeddings. Sentence Transformers are free and can run locally. OpenAI's models run through an API and are easy to use but send data outside the company.

4. Chunking

Chunking means cutting a long document into smaller pieces before storing it. This matters because embedding models and LLMs can only handle a limited amount of text at once, and smaller pieces give better search results.

Fixed-size chunking just cuts text every set number of characters. It's simple but can cut sentences in half. Recursive chunking is smarter: it tries to cut at paragraph or sentence breaks first. Semantic chunking cuts based on where the topic actually changes, which gives the best results but takes more work to set up.

Chunk overlap means letting nearby chunks share a bit of text at the edges. This stops important information from getting lost right at the border between two chunks.

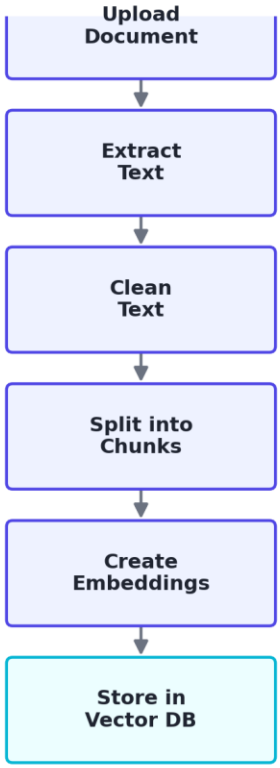


Figure 2: How a document is processed and stored.

5. Retrieval

Vector search finds stored chunks that are closest in meaning to a question, even if the wording is different. The most common way to measure this closeness is cosine similarity, which checks the angle between two vectors. A score close to 1 means very similar; a score close to 0 means unrelated.

Top-k retrieval means picking only the best few matches, like the top 4 or 5, instead of everything. This keeps the answer focused. Hybrid search adds normal keyword search on top of vector search, which helps when a question includes an exact term like a product code.

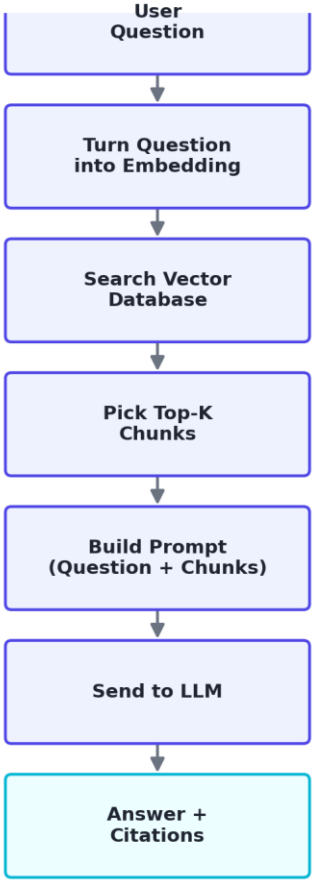


Figure 3: How a question turns into an answer.

6. Prompt Construction

Once the system finds the right chunks, it puts them together with the question into one prompt for the LLM. The prompt usually has three parts: a system message with the rules, the retrieved chunks as context, and the question itself.

The system message matters a lot. It tells the model to only use the given context, not to make things up, and to say "I don't know" if the answer isn't there. A clear prompt like this often improves answers more than using a bigger model.

7. Vector Databases

A vector database stores embeddings and searches through them fast, even with millions of entries. Popular options include ChromaDB, FAISS, Pinecone, Weaviate, Milvus, and Qdrant. ChromaDB and FAISS are common for local or smaller setups. Pinecone is a managed cloud service that handles scaling for you. Weaviate, Milvus, and Qdrant work well for both local and cloud use.

Local vector databases keep data inside the company and cost less, but need more setup. Cloud vector databases handle scaling automatically but usually cost more and send data to an outside provider.

8. Hallucination Reduction

A hallucination is when an AI gives a wrong answer that still sounds confident. RAG helps reduce this by grounding answers in real, retrieved text instead of guesses. Better retrieval means pulling only truly relevant chunks. Citations

let users check the source themselves. Clear prompt instructions stop the model from filling gaps with made-up facts. Confidence checking looks at how relevant the retrieved chunks really are, and warns the user if nothing good was found.

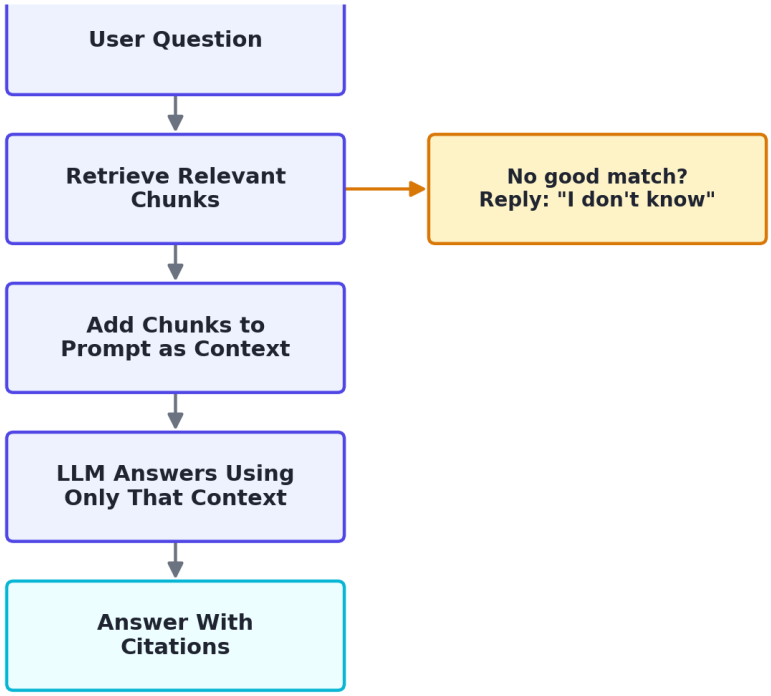


Figure 4: How RAG keeps answers grounded in real sources.

9. Future Improvements

RAG is still improving. Multi-modal RAG can search images and tables, not just text. Agentic RAG lets the AI plan several search steps instead of just one. Graph RAG and Knowledge Graphs store relationships between facts, so the system can answer more connected questions. Better reranking sorts retrieved chunks more carefully. Personalized search adjusts results per user, and real-time indexing makes new documents searchable within seconds.

10. Conclusion

Enterprise RAG lets companies use powerful AI models while staying grounded in their own accurate data. It is cheaper than retraining a model, keeps data private, and builds trust through citations. This is why more organizations keep adopting it, and why RAG is becoming a standard part of enterprise AI systems.

References

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, 9459–9474.

LangChain. (n.d.). LangChain documentation. Retrieved from https://python.langchain.com/

Sentence Transformers. (n.d.). Sentence Transformers documentation. Retrieved from https://www.sbert.net/

Chroma. (n.d.). Chroma documentation. Retrieved from https://docs.trychroma.com/

Meta AI. (n.d.). FAISS documentation. Retrieved from https://faiss.ai/

OpenAI. (n.d.). OpenAI platform documentation. Retrieved from https://platform.openai.com/docs/